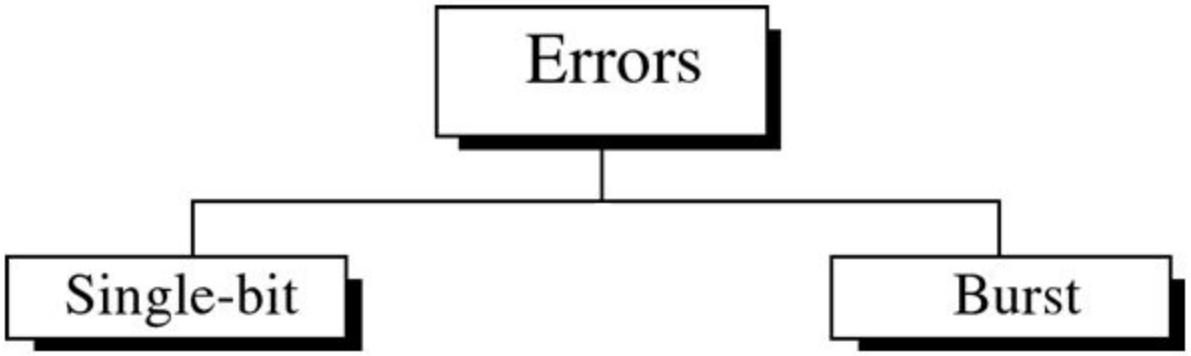
Data Communication
CSE 2211

Chapter 10 : Error Detection and Correction

Errors

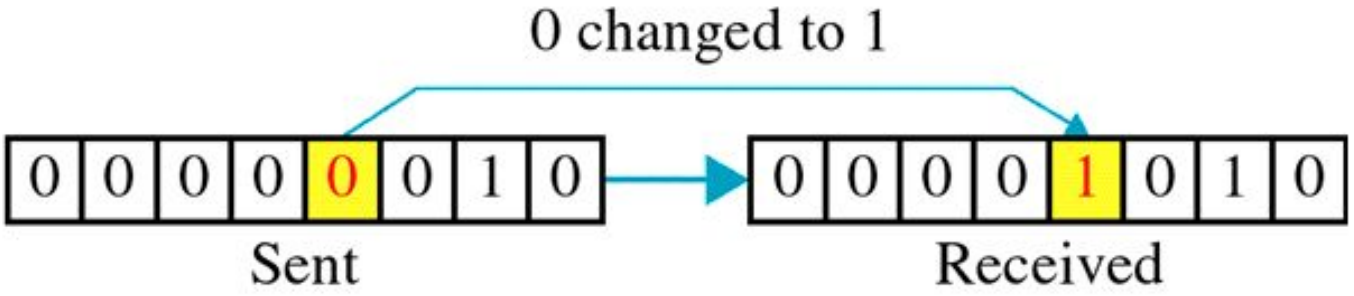
- *Data can be corrupted during transmission.*
- *For reliable communication, errors must be detected and corrected.*
- *Error detection and correction are implemented either at the data link layer or the transport layer of the OSI model.*

Types of Errors



Single bit error

- *The term single bit error means that only one bit of a given data unit (such as a byte, character, data unit, or packet) is changed 1 to 0 or from 0 to 1.*



Single bit error in Serial Transmission:

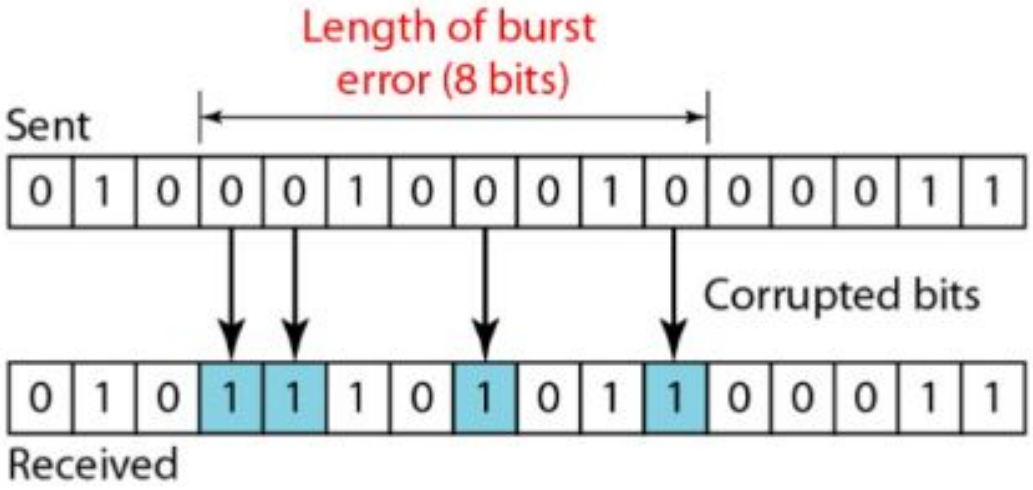
- *Single-bit errors are the least likely type of error in serial data transmission.*
- *For example, imagine a sender sends data at 1 Mbps. This means that each bit lasts only 1µs.*
- *For a single bit error to occur, the noise must have a duration of only 1 µs, which is very rare; noise normally lasts much longer than this.*

Single bit error in Parallel Transmission:

- *However, a single bit error can happen if we are sending data using parallel transmission.*
- *For example, if eight wires are used to send all of the eight bits of a byte at the same time and one of the wires is noisy, one bit can be corrupted in each byte.*

Burst Error

- *The term burst error means that two or more bits in the data unit have changed from 1 to 0 or 0 to 1.*
- *The length of the burst is measured from the first bit to the last corrupted bit.*
- *Some bits in between may not have been corrupted.*

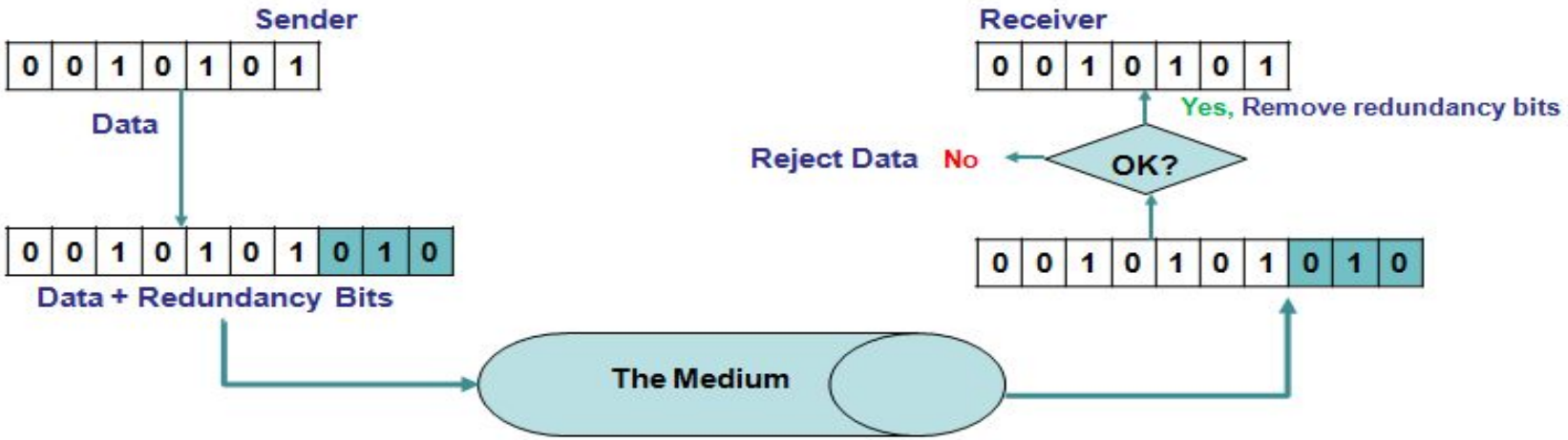


Burst Error in Serial Transmission

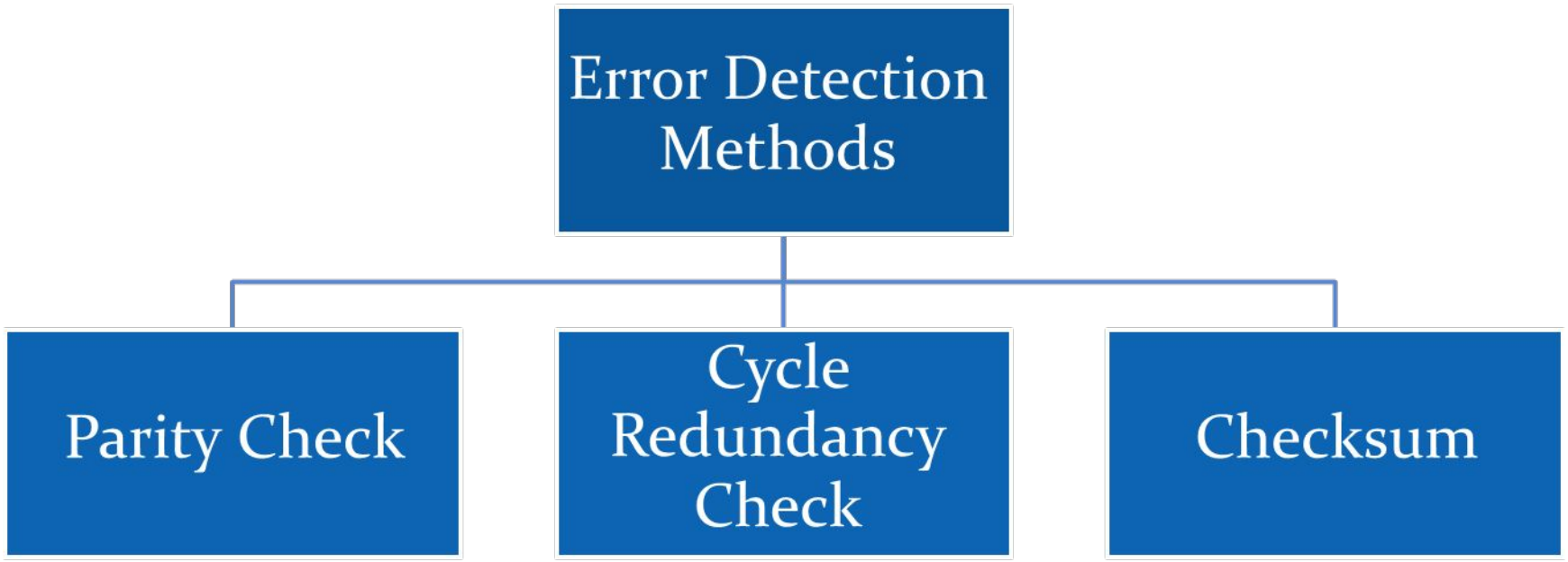
- *Burst error is most likely to happen in a serial transmission.*
- *The duration of noise is normally longer than the duration of a bit, which means that when noise effects data, it effects a set.*
- *The number of bits affected depends on the data rate and duration of noise.*
- *For example, if we are sending at at 1Kbps, a noise of 1/100 seconds can effect 10 bits; if we are sending data at 1Mps, the same noise can affect 10,000 bits.*

Error Detection

- *Error detection means to decide whether the received data is correct or not without having a copy of the original message.*
- *Error detection **uses the concept of redundancy**, **which means** adding extra bits for detecting errors at the destination.*
- *These redundant bits are added by the sender and removed by the receiver.*

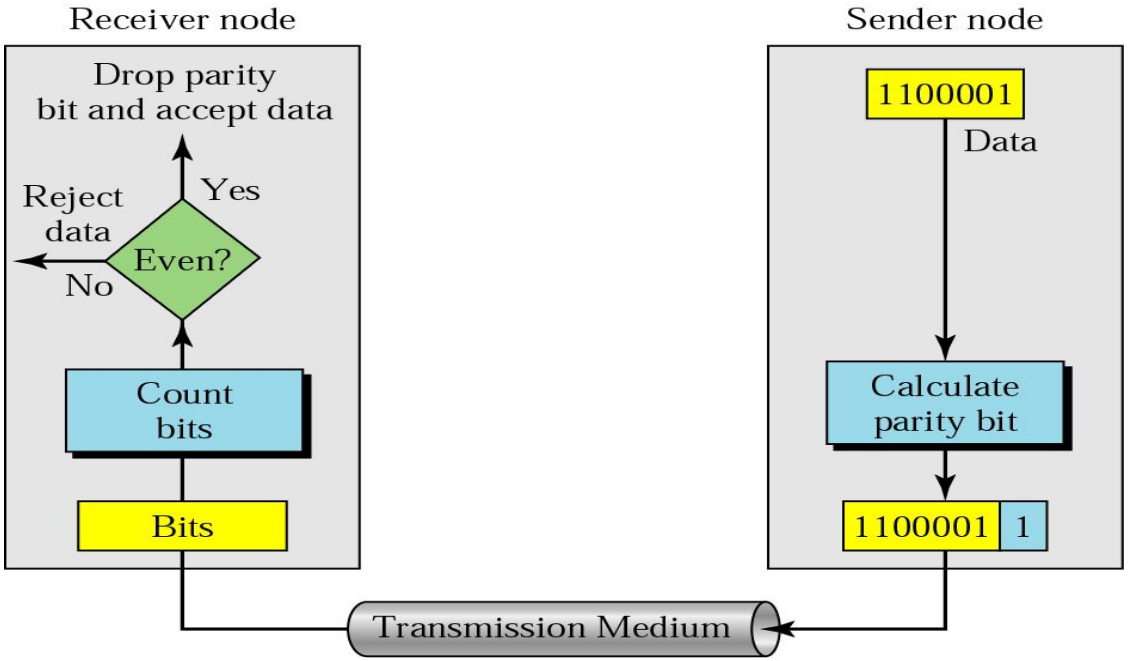


Error Detection Methods



Parity Check (Vertical Redundancy Check)

- The most common and least expensive mechanism
- Single bit is added to a block
- Two schemes:
  - Even parity – Maintain even number of 1s
    E.g., 1011 → 10111
  - Odd parity – Maintain odd number of 1s
    E.g., 1011 → 10110



Example 1

Suppose the sender wants to send the word *world*. In ASCII the five characters are coded as an 7-bit binary word

1110111 1101111 1110010 1101100 1100100

The following shows the actual bits sent

1110111**0** 1101111**0** 1110010**0** 1101100**0** 1100100**1**

Example 2

Now suppose the word world in Example 1 is received by the receiver without being corrupted in transmission.

11101110 11011110 11100100 11011000 11001001

The receiver counts the 1s in each character and comes up with even numbers (6, 6, 4, 4, 4). The data are accepted.

Example 3

Now suppose the word world in Example 1 is corrupted during transmission.

11111110 11011110 11101100 11011000 11001001

The receiver counts the 1s in each character and comes up with even and odd numbers (7, 6, 5, 4, 4). The receiver knows that the data are corrupted, discards them, and asks for retransmission.

Vertical Redundancy Check Performance

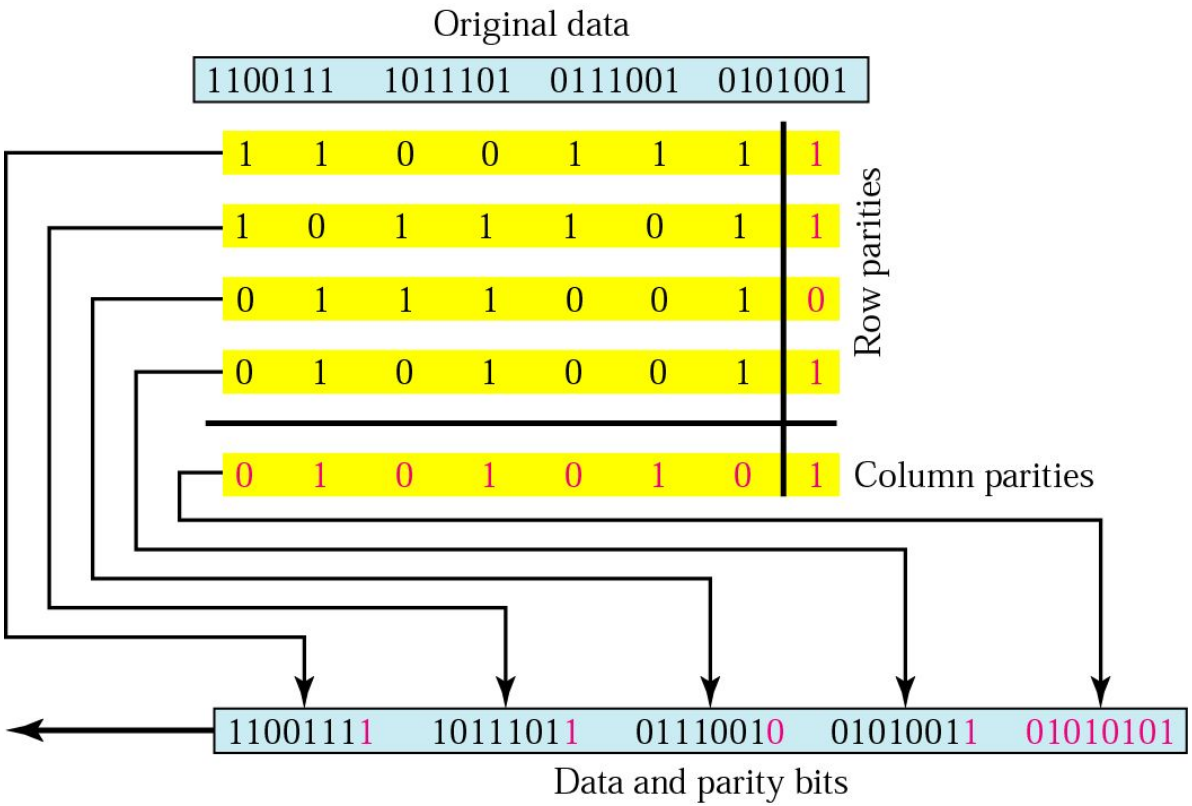
- *VRC can detect all single-bit errors.*
- *It can also detect burst errors as long as the total number of bits changed is odd (1,3,5 etc.).*
- *Let's say we have an even-parity data unit where the total number of 1s including the parity bit is 6: 1000111011.*
- *If any three bit change value, the resulting parity will be odd and the error will be detected: 1111111011:9, 0110111011:7, 1100010011:5-all odd.*
- *The VRC checker would return a result of 1 and the data unit would be rejected. The same holds true for any odd number of errors.*

Vertical Redundancy Check Performance

- *Suppose however, that two bits of the data unit are changed: 1110111011:6, 1000011010:4.*
- *In each case the number of 1s in the data unit is still even.*
- *The VRC checker will add them and return an even number although the data unit contains two errors.*
- *VRC cannot detect errors where the total number of bits changed is even.*

Two-dimensional parity

- A better approach is the Two-dimensional parity check.
- In two-dimensional parity check, a block of bits is divided into rows and a redundant row of bits is added to the whole block.



Example 4

Suppose the following block is sent:

10101001 00111001 11011101 11100111 10101010

However, it is hit by a burst noise of length 8, and some bits are corrupted.

1010**0011** **1000**1001 11011101 11100111 10101010

When the receiver checks the parity bits, some of the bits do not follow the even-parity rule and the whole block is discarded.

10100011 10001001 11011101 11100111 **1**0**101**0**1**0

Two-dimensional parity performance

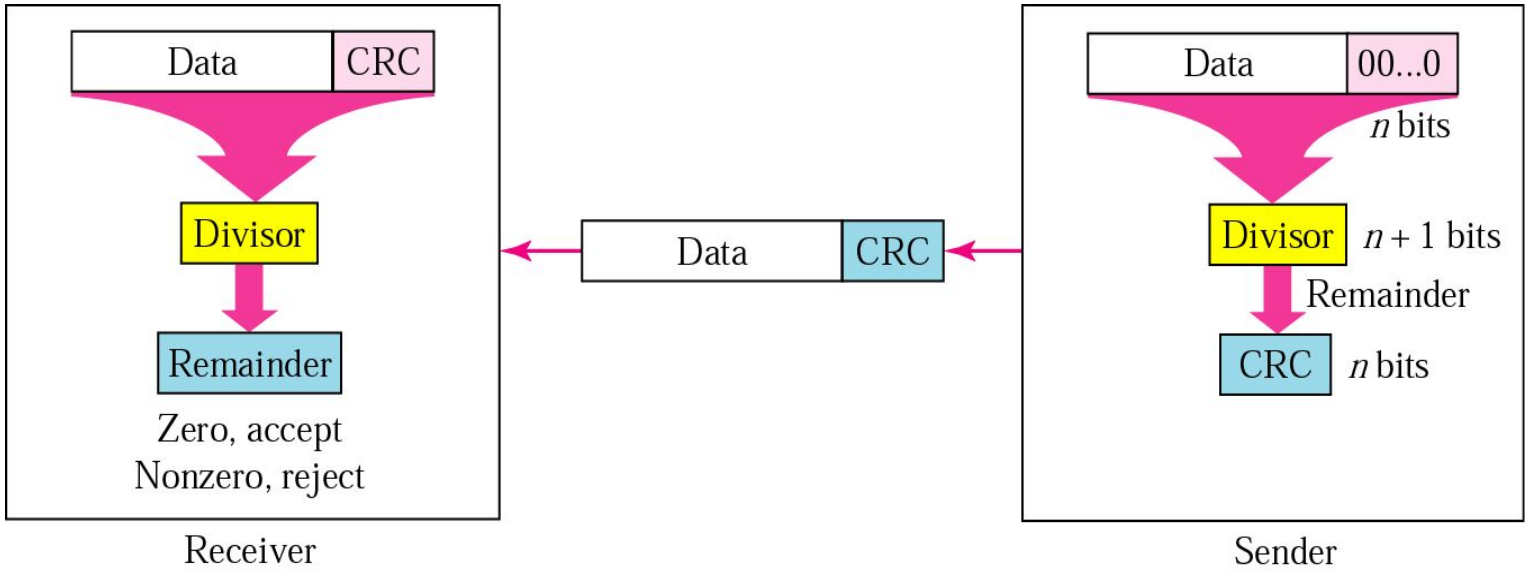
- Two dimensional parity check cannot detect burst error if -
  - Two bits in one data unit are damaged
  - Two bits in exactly the same positions in another one data unit are also damaged

Cyclic Redundancy Check (CRC)

- Used for error checking
- Unlike the parity check which is based on addition, CRC is based on binary division.
- Before sending data ,some calculation are done on the data packets to generate a CRC number
- The CRC is calculated so the packet (data + CRC) becomes exactly divisible by a predetermined number.
- When data is received , the packet is divided by the number.

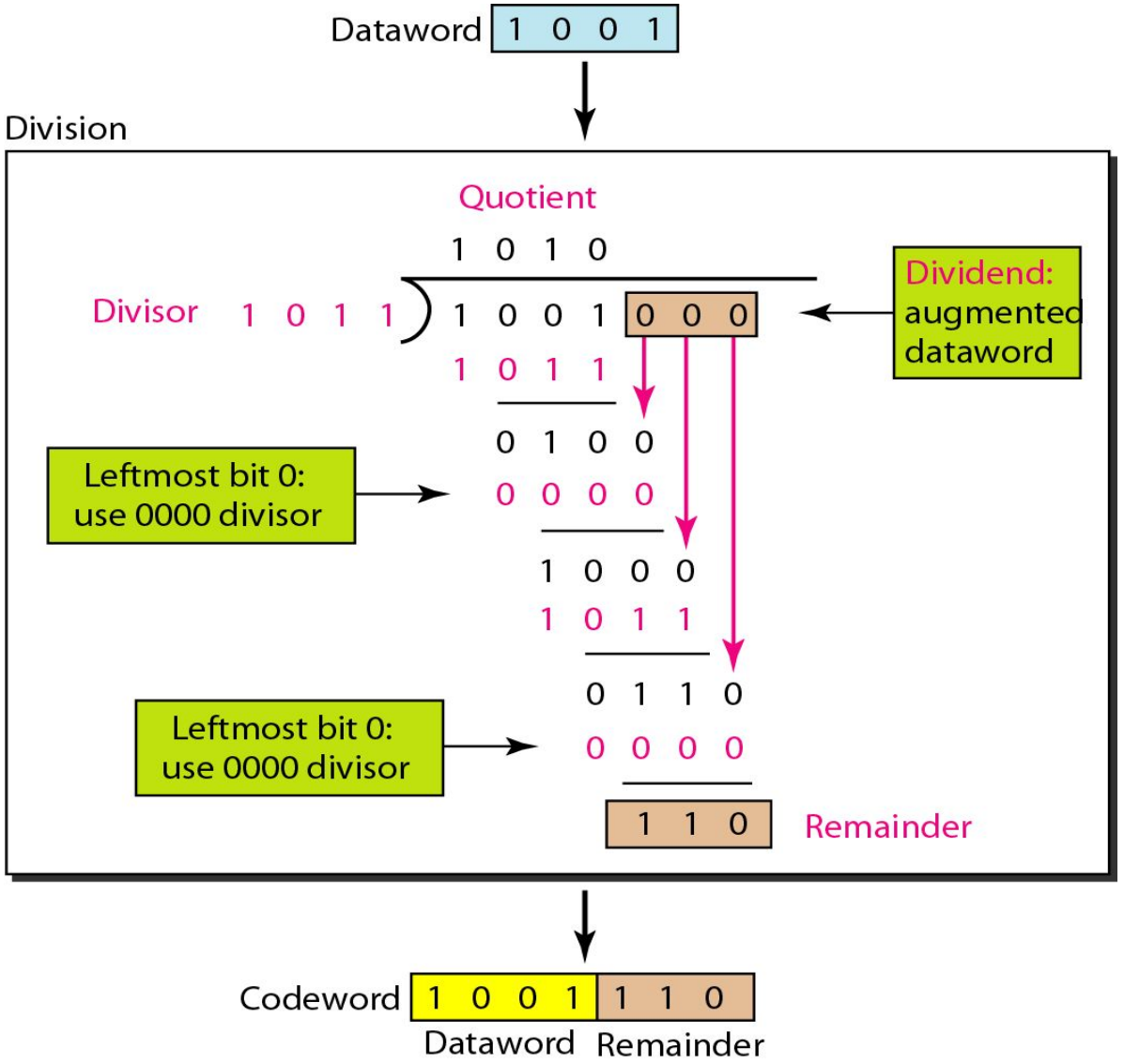
Cyclic Redundancy Check (CRC)

- If the reminder "0" : data is correct
- If the reminder "1" : data has a problem and need to be sent again.
- CRC Can detect single bit & burst errors.

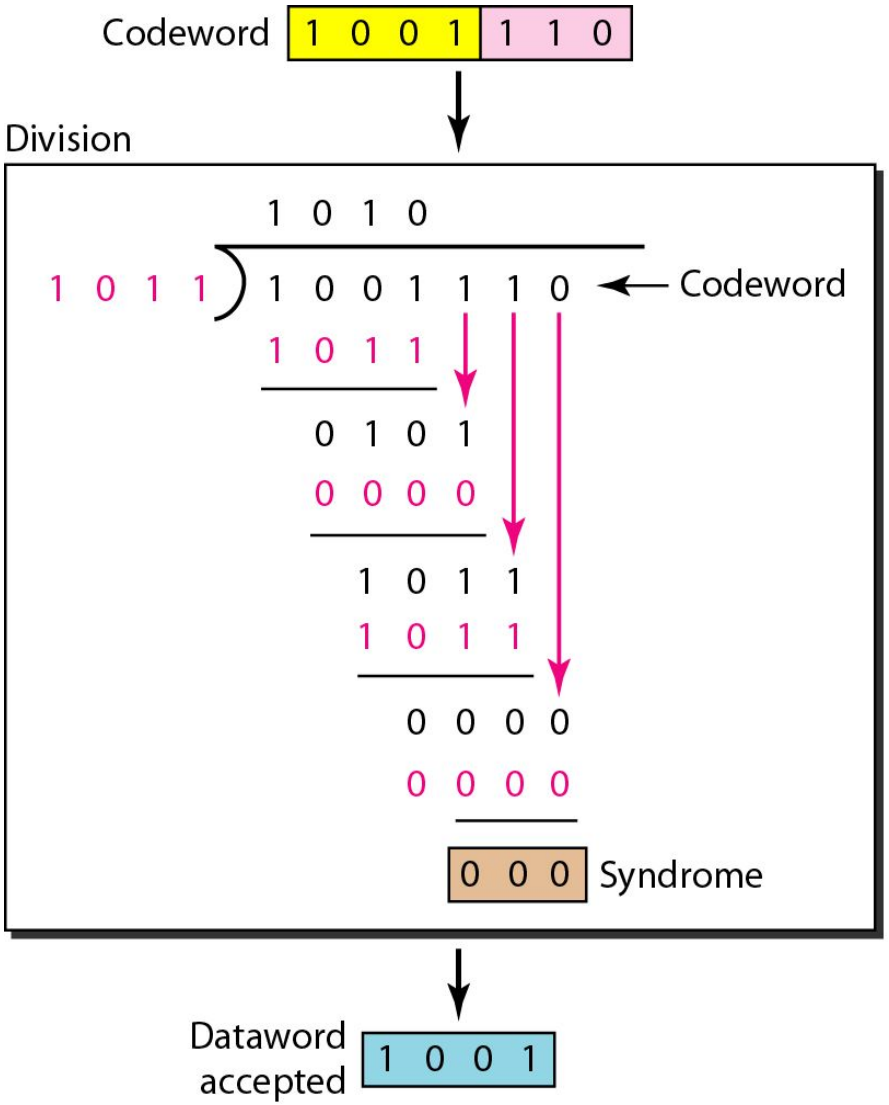


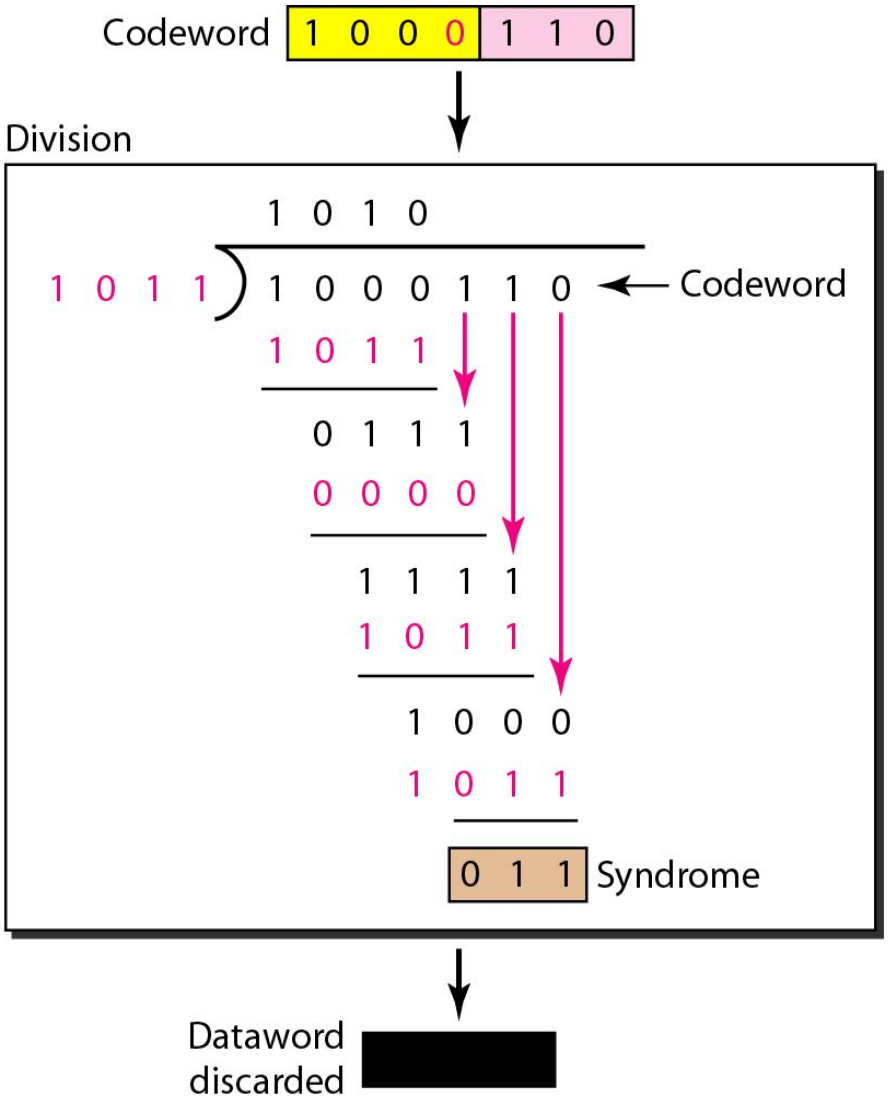
The CRC Generator (Ex-1)

The CRC generator uses modulo-2 divisions.



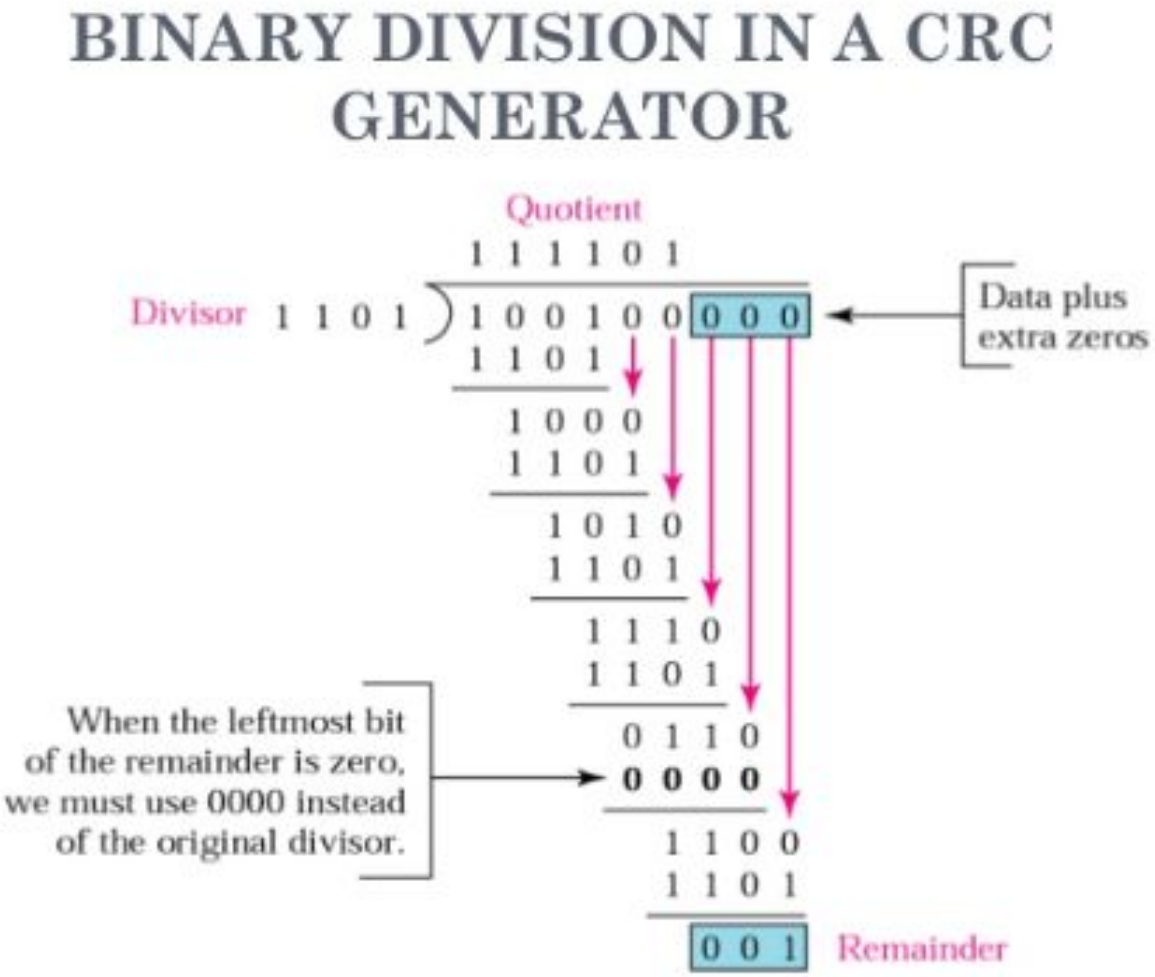
The CRC Checker (Ex-1)



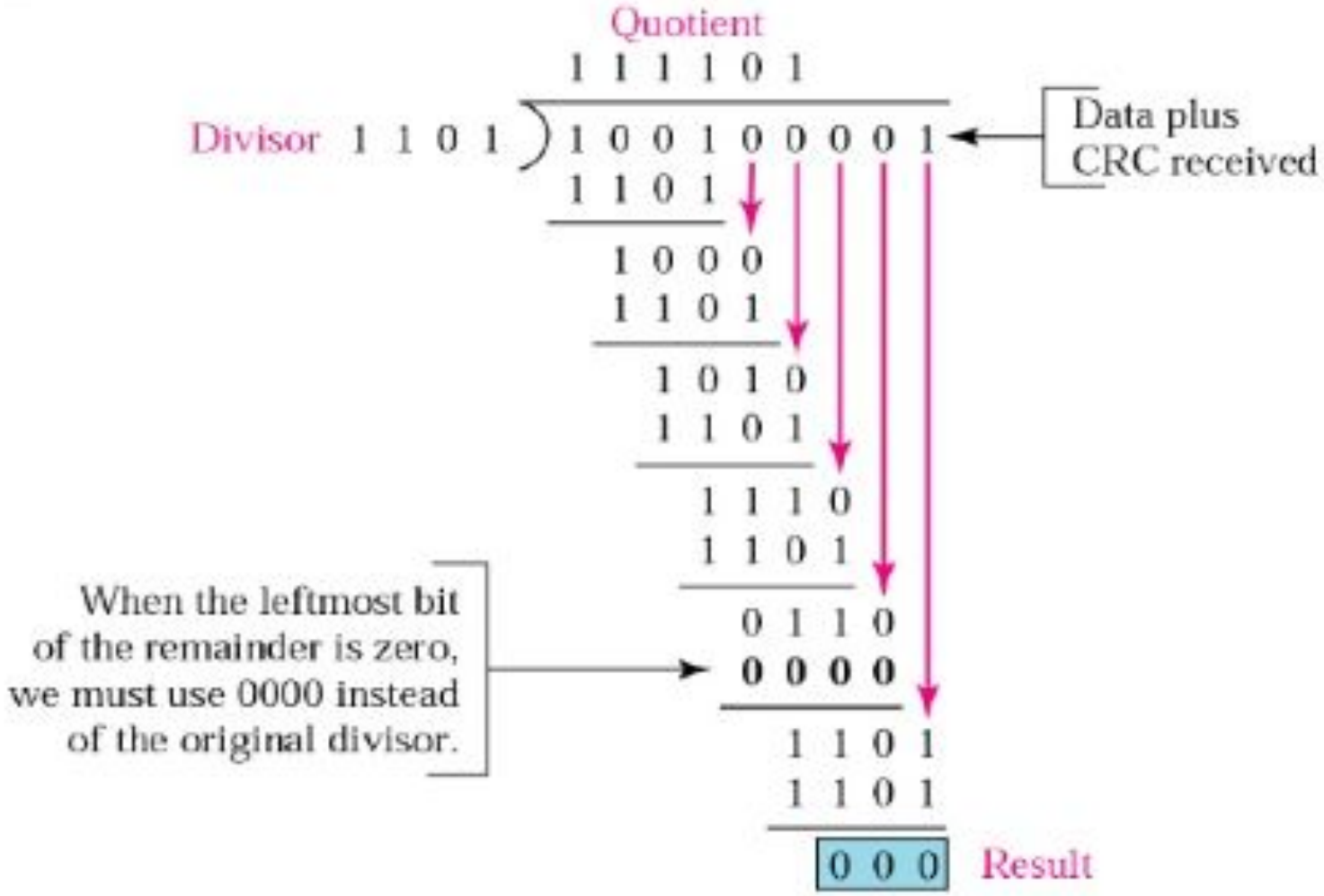


The CRC Generator (Ex-2)

The CRC generator uses modulo-2 divisions.

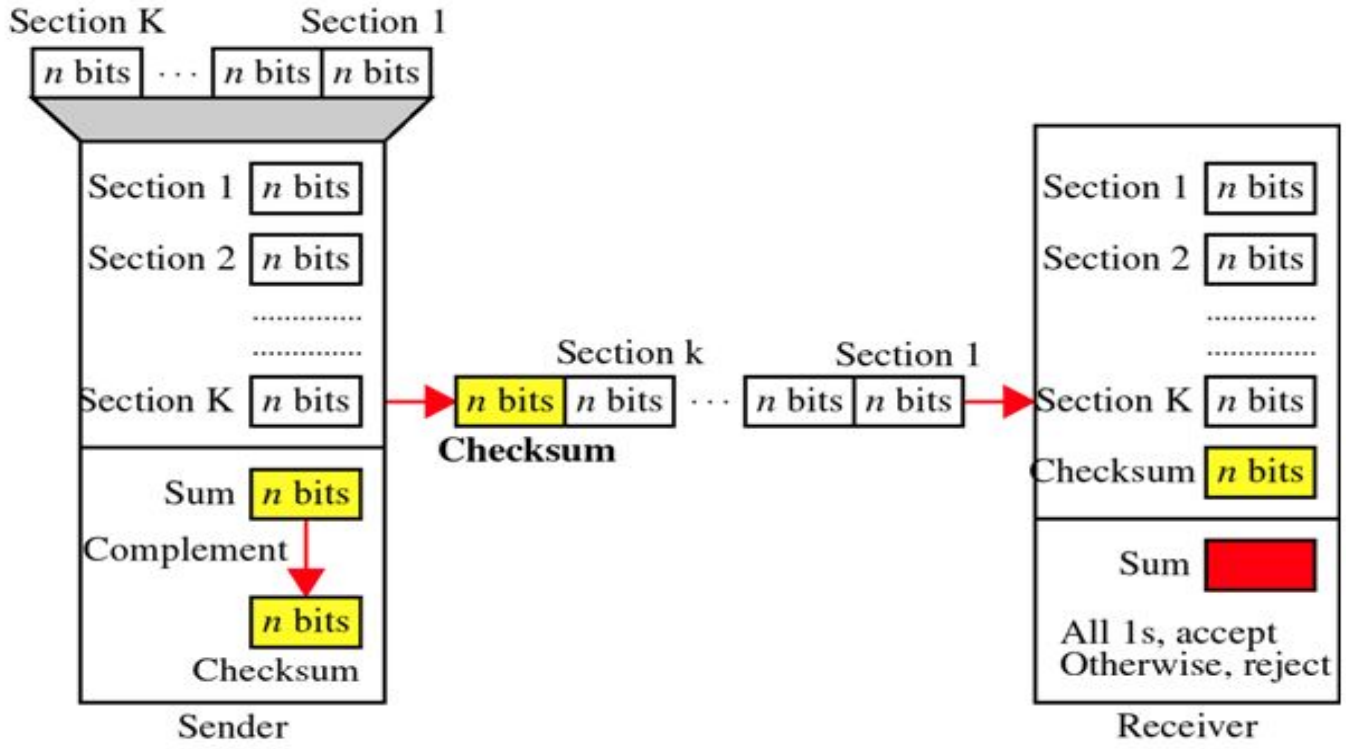


The CRC Checker (Ex-2)



Checksum

- A checksum "adds" together "chunks" of data.
  - A **match** between receiver checksum and transmitted checksum indicates good data.
  - A **mismatch** indicates an error has occurred.
- Capable of detecting single or burst errors.
- It is simple to implement in either hardware or software.



Checksum-at the sender

- *The unit is divided into k sections, each of n bits.*
- *All sections are added together using one's complement to get the sum.*
- *The sum is complemented and becomes the checksum.*
- *The checksum is sent with the data*

Checksum-at the receiver

- *The unit is divided into k sections, each of n bits.*
- *All sections are added together using one's complement to get the sum.*
- *The sum is complemented.*
- *If the result is zero, the data are accepted: otherwise, they are rejected.*

Checksum-example-1

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits.

10101001 00111001

The numbers are added using one's complement

	10101001
	00111001

Sum	11100010
Checksum	00011101

The pattern sent is 10101001 00111001 00011101

Checksum-example-1

Now suppose the receiver receives the pattern sent in and there is no error.

10101001 00111001 00011101

When the receiver adds the three sections, it will get all 1s, which, after complementing, is all 0s and shows that there is no error.

	10101001
	00111001
	00011101
Sum	11111111
Complement	**00000000** means that the pattern is OK.

Checksum-example-1

Now suppose there is a burst error of length 5 that affects 4 bits.

10101111 11111001 00011101

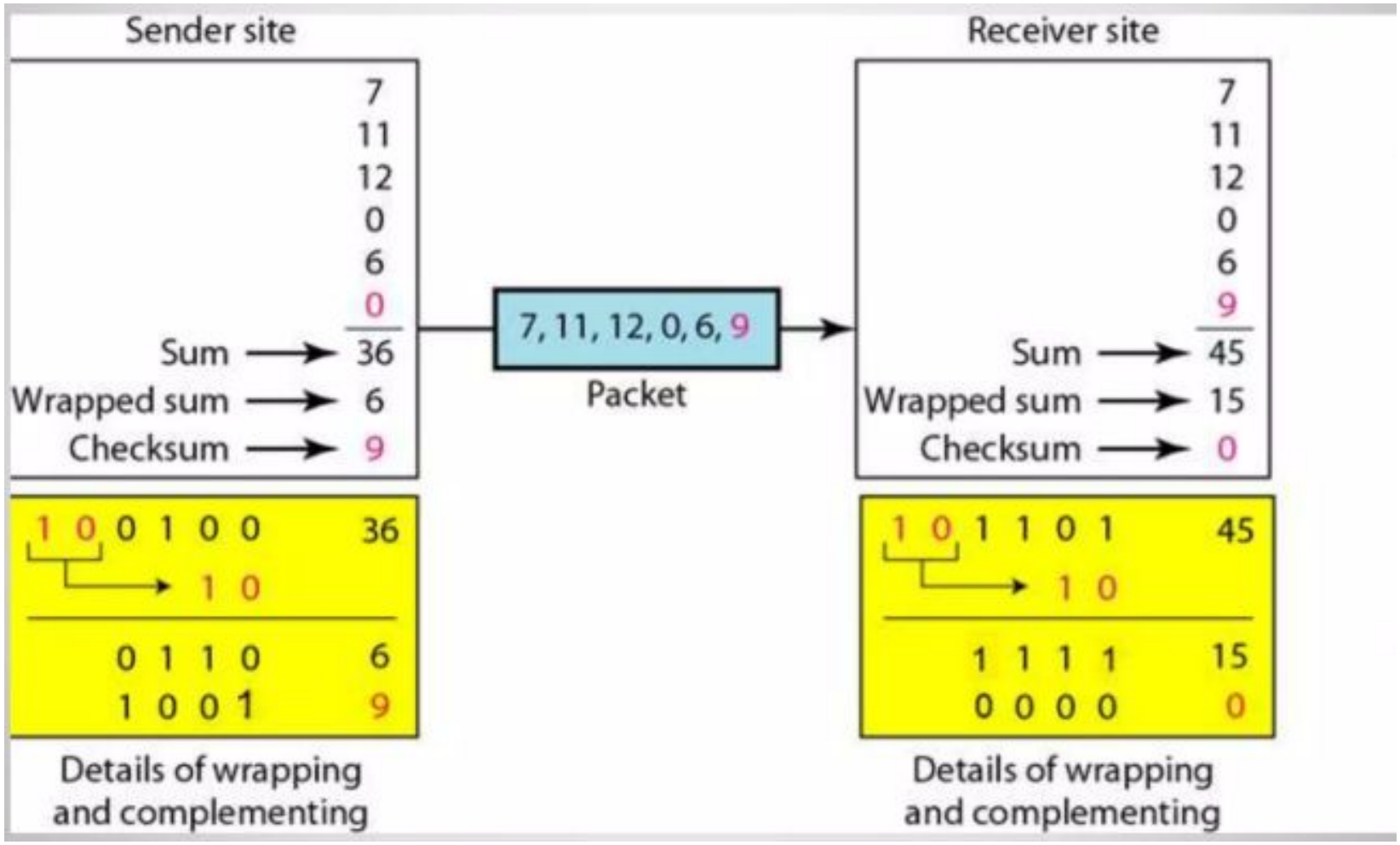
When the receiver adds the three sections, it gets

	10101111	
	11111001	
	00011101	
Partial Sum	1 11000101	
Carry	1	
Sum	11000110	
Complement	00111001	the pattern is corrupted.

Example- 2

Suppose our data is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers. For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum. If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum. Otherwise, there is an error somewhere and the data are not accepted.

Example- 2


Sender site
7
11
12
0
6
0
Sum → 36
Wrapped sum → 6
Checksum → 9
1 0 0 1 0 0 36
1 0
0 1 1 0 6
1 0 0 1 9
Details of wrapping and complementing
7, 11, 12, 0, 6, 9
Packet
Receiver site
7
11
12
0
6
9
Sum → 45
Wrapped sum → 15
Checksum → 0
1 0 1 1 0 1 45
1 0
1 1 1 1 15
0 0 0 0 0
Details of wrapping and complementing

Error Correction

Introduction

- *In error correction, we need to know the exact number of bits that are corrupted and, more importantly, their location in the message.*
- *Error can be handled in two ways:*
  - *Receiver can have the sender retransmit the entire data unit.*
  - *The receiver can use an error-correcting code, which automatically corrects certain errors. (Forward error correction (FEC).*

Forward Error Correction

- *Consider only a single-bit error in k bits of data*
  - *k possibilities for an error*
  - *One possibility for no error*
  - *possibilities = k + 1*
- *Add **r** redundant bits to distinguish these possibilities; we need*

$$2^r \geq k+1$$

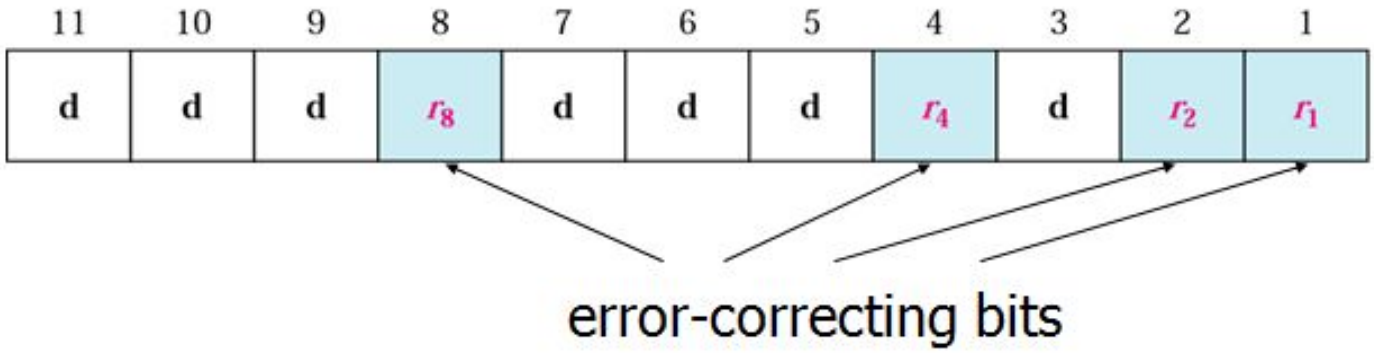
- *But the **r** bits are also transmitted along with data; hence*

$$2^r \geq k+r+1$$

Number of data bits *k*	Number of redundancy bits *r*	Total bits *k + r*
1	2	3
2	3	5
3	3	6
4	3	7
5	4	9
6	4	10
7	4	11

Hamming Code

- *Simple, powerful FEC*
- *Widely used in computer memory*
- *Positions of redundancy bits in Hamming code*

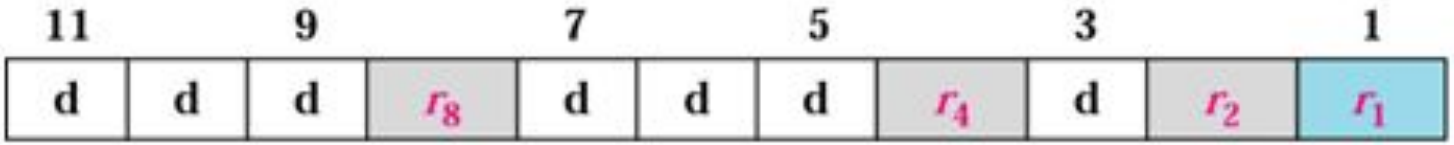


Calculating Hamming Code

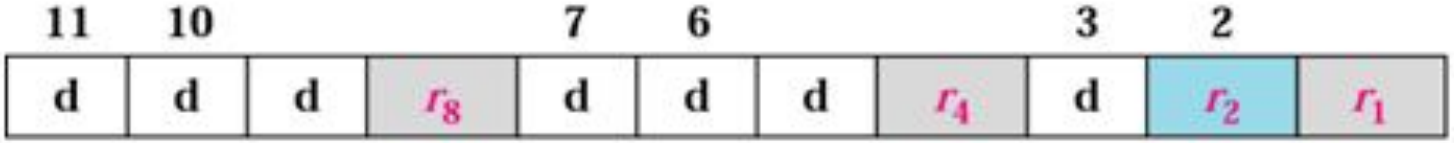
- The key to the Hamming Code is the use of extra parity bits to allow the identification of a single error. Create the code word as follows:
- Mark all bit positions that are powers of two as parity bits. (positions 1, 2, 4, 8, 16, 32, 64, etc.)
- All other bit positions are for the data to be encoded. (positions 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 17, etc.)
- Each parity bit calculates the parity for some of the bits in the code word. The position of the parity bit determines the sequence of bits that it alternately checks and skips.
- Position 1: check 1 bit, skip 1 bit, check 1 bit, skip 1 bit, etc. (1,3,5,7,9,11,13,15,...)
- Position 2: check 2 bits, skip 2 bits, check 2 bits, skip 2 bits, etc. (2,3,6,7,10,11,14,15,...)
- Position 4: check 4 bits, skip 4 bits, etc. (4,5,6,7,12,13,14,15,...)
  Position 8: check 8 bits, skip 8 bits, etc. (8-15,24-31,40-47,...)
- Set a parity bit to 1 if the total number of ones in the positions it checks is odd. Set a parity bit to 0 if the total number of ones in the positions it checks is even.

Redundant Bit Calculation

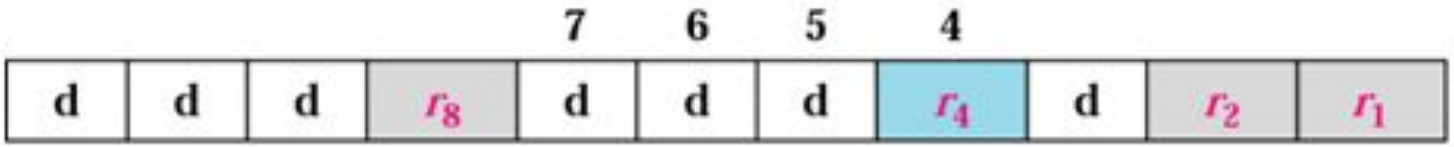
r_1 will take care of these bits.



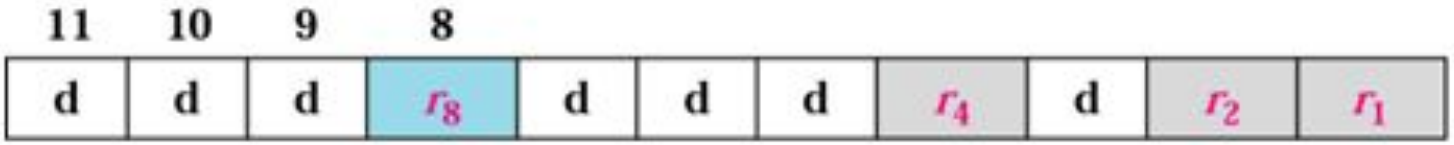
r_2 will take care of these bits.



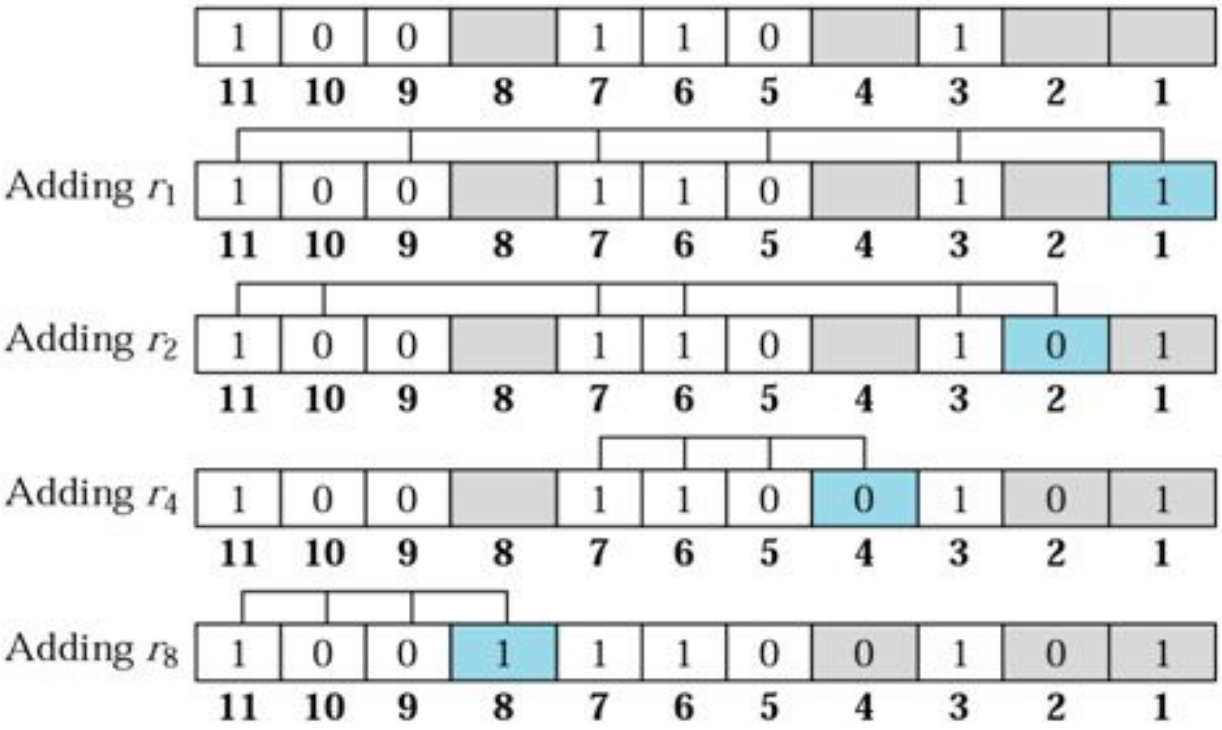
r_4 will take care of these bits.



r_8 will take care of these bits.



Redundant Bit Calculation



Data:
1 0 0 1 1 0 1

Code:
1 0 0 1 1 1 0 0 1 0 1

Redundant Bit Calculation

- Receiver receives 10010100101

